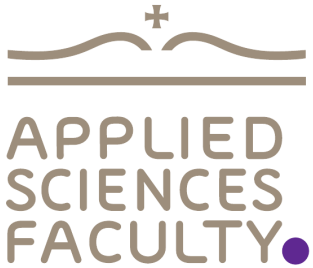
C++ – допоміжні інструменти, C++11/14/17/20/23/26.





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

tuple — кортежі

- std::tuple – кортеж, оголошено в <tuple>. Гетерогенна колекція фіксованого розміру.

```
namespace std {
template <typename... Types>
class tuple;
}
```

Як він може бути реалізованим, ми вивчали, говорячи про variadic templates.

- Ітерувати по кортежу не можна[1], для доступу до елементів служить функція get<N>(tpl):

```
tuple<double, int, string> t2{5.7, 3, "anc"};
cout << get<0>(t2) << ", " << get<1>(t2) << ", " << get<2>(t2);
```

- Можуть зберігати посилання:

```
string str1{"xyz"};      double data = 3.14;      int val = 3;
tuple<double&, int&, string&> t6{data, val, str1};
cout << str1 << " ... " << get<2>(t6) << endl;
get<2>(t6) = "abc";
cout << str1 << " ... " << get<2>(t6) << endl;
```

[1] Це C++ – тому, з використанням шаблонного метапрограмування – можна. Див., наприклад https://blog.tartanllama.xyz/exploding-tuples-fold-expressions/.

Подробиці

- Неініціалізовані члени ініціалізуються конструктором за замовчуванням. Зокрема[2], фундаментальні типи ініціалізуються нулем.
- make_tuple для створення кортежу посилань можна використати так:

```
auto t7 = std::make_tuple(ref(data), ref(val), ref(str1));
```

- Крім звертання за номером, можна також звертатися за типом:

```
tuple<double, int, string> t{5.7, 3, "anc"};
cout << get<double>(t) << ", " << get<int>(t) << ", " << get<string
```

- Якщо тип не унікальний – буде помилка компіляції. Те ж буде, якщо звернутися до номера, якого не існує.
- make_tuple для стрічкових літералів C виведе тип const char*, щоб отримати std::string слід скористатися його літералом:

```
using namespace std::literals::string_literals;
auto t3 = std::make_tuple(9.11, 5, "test");
auto t4 = std::make_tuple(9.11, 5, "test"s);
cout << get<const char*>(t3) << endl; // get<string>(t3) -- Error!
cout << get<string>(t4) << endl;
```

[2] Що не типово для C/C++!

Порівняння

- Для кортежів є всі оператори порівняння.
- Порівняння лексикографічне.
- Оскільки оператори шаблонні, можна порівнювати кортежі із різними типами, однак, кількість елементів має збігатися.

Інше

- Існує конструктор, який приймає std::pair.
- Аналогічно, існує конструктор std::pair, який приймає кортеж із двох елементів.
- Існують також конструктори, яким можна передати користувацький алокатор для створення елементів кортежу.
- tuple_cat() дозволяє об'єднати кілька кортежів:

- Sidenote: див. також `https://stackoverflow.com/a/32999125` – про conditional explicite конструктори.
- `std::tuple_size<type>::value` дозволяє отримати розмір кортежу:

```
cout << tuple_size<decltype(make_tuple(9.11, 5, "test"))>::value;

using tuple_type_1 = decltype(make_tuple(9.11, 5, "test"));
cout << tuple_size<tuple_type_1>::value << endl;
```

- Ергономічність, правда, дискусійна... C++17 має більш зручний синонім: `tuple_size_v<type>`:

```
cout << std::tuple_size_v <tuple_type_1> << endl;
```

- `std::tuple_element<idx, type>::type` дозволяє отримати тип елементу із номером idx.
- `std::tuple_cat()` дозволяє об'єднувати довільну кількість кортежів в один:

```
auto cres = std::tuple_cat(c1, c2);
cout << get<int>(cres) << ", ";
cout << get<char>(cres) << ", ";
cout << get<double>(cres) << ", ";
cout << get<float>(cres) << std::endl;
```

- **template**< **class**... Types >
  tuple<Types&&...> forward_as_tuple(Types&&... args) noexcept;

  створює кортеж посилань, зручних для perfect forwarding. Увага – не продовжує часу життя переданих аргументів.
- **template**< **class**... Types >
  tuple<Types&...> tie(Types&... args) noexcept;

  Дозволяє створювати кортеж посилань на аргументи.
- std::ignore вказує пропускати даний аргумент.
- std::tie може використовуватися для розпакування кортежів чи пар, завдяки відповідному конструктору std::tuple.
- Приклад:

```
auto fn1 = [](){ return std::make_tuple(2, 3.14, "abc"s); };
auto fn2 = [](){ return std::make_pair(11, 22); };
int a, b;          double c;          std::string s;
std::tie(a, c, s) = fn1();
std::tie(a, std::ignore, s) = fn1();
std::tie(a, c, std::ignore) = fn1();
std::tie(std::ignore, c, s) = fn1();
std::tie(std::ignore, std::ignore, s) = fn1();
std::tie(a, b) = fn2();
std::tie(a, std::ignore) = fn2();
```

- Стандарт інструментів вводу-виводу не передбачає, але їх легко створити самостійно:

```
template<class Tuple, std::size_t N>
struct tuple_printer {
    static void print(std::ostream& os, const Tuple& t){
        tuple_printer<Tuple, N-1>::print(os, t);
        os << ", " << std::get<N-1>(t);
    }
};

template<class Tuple>
struct tuple_printer<Tuple, 1>{
    static void print(std::ostream& os, const Tuple& t){
        os << std::get<0>(t);
    }
};

template<class... Args>
std::ostream& operator<<(std::ostream& os, const std::tuple<Args...>&
    os << "[";
    tuple_printer<decltype(t), sizeof...(Args)>::print(os, t);
    os << "]\n";
```

- В цілому, ймовірно, краще використовувати структури, а не кортежі – останні потрібні хіба у випадках роботи з шаблонами з змінною кількістю аргументів.
- Цікава серія статей: "Stupid std::tuple tricks" – об'єднання, розділення і все таке:
- `https://devblogs.microsoft.com/oldnewthing/20200622-00/?p=103880` – "Mundane std::tuple tricks: Getting started",
- `https://devblogs.microsoft.com/oldnewthing/20200623-00/?p=103883` – "Mundane std::tuple tricks: Selecting via an index sequence",
- `https://devblogs.microsoft.com/oldnewthing/20200624-00/?p=103886` – "Mundane std::tuple tricks: Selecting via an index sequence, part 2",
- `https://devblogs.microsoft.com/oldnewthing/20200625-00/?p=103890` – "Mundane std::tuple tricks: Creating interesting index sequences",
- `https://devblogs.microsoft.com/oldnewthing/20200626-00/?p=103896` – "Mundane std::tuple tricks: Creating more interesting index sequences",
- `https://devblogs.microsoft.com/oldnewthing/20200629-00/?p=103910` – "Mundane std::tuple tricks: Finding a type in a tuple",
- `https://devblogs.microsoft.com/oldnewthing/20220428-00/?p=106540` – "What's up with std::piecewise_construct and std::forward_as_tuple?".

boost::tuple

- Кортежі в boost є дуже давно – власне там апробовувалися для стандартизації.
- Хедери: <boost/tuple/tuple.hpp>, <boost/tuple/tuple_comparison.hpp>, <boost/tuple/tuple_io.hpp>.
- Відмінності – незначні, однак boost::tuple містять також методи t.get<N>(), тоді як std::tuple – лише незалежні функції get<N>(t).
- Є такі інструменти: element<N, T>::type, length<T>::value замість згадуваних вище.
- Крім того, для boost::tuple реалізовано оператори вводу-виводу. Однак. читання стрічок може погано працювати, через можливі неоднозначності.
- Маніпулятори set_open(char) ('('), set_close(char) (')'), set_delimiter(char) (' '), дозволяють налаштувати форматування виводу. Оголошені вони в просторі імен boost::tuples.
- Про перетворення між ними: `https://stackoverflow.com/a/29224175` (мова про перетворення об'єктів відповідних типів).

- Функція std::get<>() є перевантаженою, а класи std::tuple_element і std::tuple_size спеціалізованими, окрім кортежів (std::tuple) також для std::pair та std::array[3].
- std::tuple_element для std::pair містить тип її першого чи другого елементу, std::tuple_size – завжди рівний два, а std::get<>() дає уніфікований спосіб звертатися до її елементів.
- std::tuple_element для std::array містить тип елементів масиву, std::tuple_size – його розмір, std::get<>() дає уніфікований спосіб звертатися до її елементів.
- Результатами їх роботи є, відповідно, звертання чи константи часу компіляції.

[3] std::get<>() також перевантажена для std::variant, про неї поговоримо у відповідному розділі.

Інтерфейс кортежів для агрегатів – C++26

- В C++26 запропоновано додати std::get та std::tuple_size для структур-агрегатів:

```
struct auth_info_aggreagte {
    std::int64_t id;
    std::int64_t session_id;
    std::int64_t source_id;
    std::time_t valid_till;
};

.....

auth_info_aggreagte res = .....;
auto id = std::get<0>(res);
```

- Детальніше див. "Aggregates are named tuples".
- "C++ Templates: How to Iterate through std::tuple: C++26 Packs and Expansion Statements ".

vocabulary types

- Велика група важливих типів – "**vocabulary types**", словникові типи.
  - ◇ Це типи, призначених служити інтерфейсом між різними частинами коду.
  - ◇ Стрічки чи навіть int – в якомусь сенсі теж словникові типи.
- Далі ми розглянемо колекцію більш спеціалізованих словникових типів сучасного C++: std::optional, std::any, std::variant, std::string_view, std::expected тощо.
- Детальніше про саму ідею див. "The Ecosystem Expense of Vocabulary Types".

std::optional – C++17

- std::optional може містити або не містити значення.
- Використовується як результат функції, котра може не повернути осмислене значення, але немає (простого) способу виділити одне із значень типу, який вона має повертати, як ознаку помилки[4].
- Ще одне використання – змінні чи члени класу, для яких існує коректна та очікувана ситуація, коли в них немає коректного значення.
- Також, може бути корисним для подвійної ініціалізації, коли відповідний тип не допускає конструювання за замовчуванням або воно є дорогим.
- Це свого роду монада MayBe в C++. Значно більш читабельний та безпечний варіант, ніж альтернативи типу std::pair<T, bool> чи виділення котрогось із значень інтегрального типу як ознаки помилки.
- Однак, використовувати його слід лише тоді, коли є виключно один, очевидний, привід не мати осмисленого результату і така ситуація теж є природною.

[4]Наприклад, для вказівників є nullptr, але коли atol() повертає 0, це може означати як те, що у вхідній стрічці був нуль, так і те, що вона не містила коректного числа – всі значення long int означають просто число.

- Приклад:

```
#include <optional>

std :: optional<double> safe_sqrt(double x){
    if( x >= 0)
        return std :: optional<double>{sqrt(x)};
    else
        return std :: optional<double>{};
}
        .....
    auto r1 = safe_sqrt(4);
    if(r1)
        cout << *r1 << endl;
    else
        cout << "No result" << endl;
```

- Альтернативою до повертання сконструйованого за замовчуванням std::optional, є повернути спеціально для цього призначену константу std::nullopt:

```
std :: optional<double> safe_sqrt(double x){
    if( x >= 0)
        return std :: optional<double>{sqrt(x)};
    else
        return std :: nullopt ;
}
```

Такий код видається читабельнішим.

- Завдяки виведенню типів C++17, можна піти ще далі:

```
std :: optional<double> safe_sqrt(double x){
    if( x >= 0)
        return std :: optional{sqrt(x)};
    else
        return std :: nullopt ;
}
```

- Ще одним класичним прикладом є функція пошуку значення в колекції:

```
template<typename Iter, typename T>
std :: optional<Iter> ofind( Iter  beg,  Iter  end, const T& val){
    auto res_iter  = std :: find (beg,  end,  val );
    if (  res_iter  != end )
        return  std :: optional{ res_iter };
    else
        return  std :: nullopt ;
}

    ..........................
    std :: vector<int> v1{3,4,5,6,7,8};
    auto f1  = ofind(v1.begin (),  v1.end (),  5);
    if (f1)
        cout  << **f1 << endl;
    else
        cout  << "No result" << endl;
```

Конструювання

- Конструктор за замовчуванням створює порожній optional. Те ж робить ініціалізація std::nullopt.

```
optional<double> od1;
optional<double> od2 = std::nullopt;
optional<double> od3{std::nullopt};
```

- Можна ініціалізувати безпосередньо:

```
optional<double> od4 = 3.14;
optional<double> od5{4.26};
optional  od6{5.37};
```

Як видно, при цьому працює автоматичне виведення аргументів шаблону.

- Існує відповідна make_optional функція для створення об'єкта, що зберігатиметься:

```
auto ocd1 = std::make_optional<std::complex<double>>(3, 4);
auto os1 = std::make_optional<std::string>("abc");
auto os2 = std::make_optional("abc"s);
```

Іноді їй доводиться підказувати, а який, власне, тип вона має вивести.

- На додачу до make_optional, std::optional має конструктор, який приймає std::in_place першим аргументом та створює об'єкт безпосередньо на тому місці, де він "житиме":

```
std::optional<std::complex<double>> oc1{std::in_place, 3, 4};
std::optional<std::vector<int>> ov1(std::in_place, {1, 2, 3, 4, 5});
```

- Ремарка – можлива реалізація std::optional могла б виглядати якось так[5]:

```
template <typename T>
class optional{
  bool _initialized;
  alignas(T) std::byte _storage[sizeof(T)];
public:
  ..............
};
```

При in_place використовується placement new.

- Реалізовано також конструктор копіювання та оператор присвоєння.

[5] До C++23 пропонувалося: "std::aligned_storage_t<sizeof(T), alignof(T)> _storage;"

Розміри std::optional

- Гарантується, що сам по собі std::optional не виділятиме динамічну пам'ять.
- Однак, через вирівнювання, розмір його може бути більшим за розмір T більше ніж на один байт (наприклад, на 4).
- GCC 8.3, MinGW; GCC 7.4, Linux; Clang 8.0, MinGW

```
char                    :  1   -- 2
int                     :  4   -- 8
float                   :  4   -- 8
double                  :  8   -- 16
std::complex<float>     :  8   -- 12
std::complex<double>    :  16  -- 24
std::vector<int>        :  24  -- 32
std::map<int, string>   :  48  -- 56
```

- Для MSVC 2017 – x64, два останні рядки відрізняються, але накладні витрати ті ж.

```
std::vector<int>        :  32  -- 40
map<int, string>        :  24  -- 32
```

- Тому, якщо кілька значень існують або не існують одночасно, загортати кожен із них в std::optional може бути нераціональним. Тоді краще або створити структуру із ними і її зберігати в optional, або реалізувати відповідний функціонал вручну.

Доступ до збереженого значення – 1

- Для перевірки, чи містить об'єкт значення, можна використати has_value(), а завдяки явному приведенню в bool, об'єкти std::optional можна безпосередньо використовувати в умовних виразах:

```
std :: optional<int> opt_val;
    ...................
if (opt_val.has_value())     {.................}
if (opt_val)     {.................}
```

- Перевантажено оператори * та ->:

```
cout << *opt_val;
*opt_val = 2;
std :: optional<std:: string > opt_str{"abc"};
cout << opt_str->size();
```

- Таким способом стан навіть вкрасти можна:

```
auto taken = *std::move(opt_str);
std :: cout << "taken: " << taken << " opt_str: "
            << *opt_str << "size: " << opt_str->size()  << endl;
```

- Використання * чи -> до порожнього об'єкту – невизначена поведінка!

Доступ до збереженого значення – 2

- Метод value() повертає посилання на вміст, або кидає `std::bad_optional_access`, якщо об'єкт порожній.
- Інший зручний метод – value_or(), який повертає значення для об'єкту, що його має, або вказане значення за замовчуванням, якщо він порожній:

```
optional<double> odA;
optional<double> odB{13};
cout << odA.value_or(3) << endl;
cout << odB.value_or(4) << endl;
```

Зміна вмісту std::optional

- Як вже згадувалося, існує оператор присвоєння.
- Метод emplace() створює новий об'єкт на місці старого. Для останнього викликається деструктор.

```
std :: optional<std:: string > ostr3;
cout << ostr3.value_or("@Empty") << endl;
ostr3.emplace("Yest");
cout << ostr3.value_or("@Empty") << endl;
ostr3.emplace("ABC");
cout << ostr3.value_or("@Empty") << endl;
```

- reset() очищає об'єкт, викликаючи за потреби деструктор збереженого об'єкта. Той же ефект, що й присвоєння std::nullopt;
- Метод swap() обмінює два std::optional місцями, є відповідна спеціалізація std::swap().

Порівняння

- Реалізовано всі операції порівняння, якщо вони підтримуються типом, збереженим в std::optional.
- Якщо обидва std::optional містять значення, вони порівнюються відповідними операторами.
- Якщо обидва std::optional не містять значення, вони теж вважаються рівними. Якщо ж значення містить тільки один – вони не рівні.
- Якщо один із std::optional містить значення, а інший ні, то перший вважається більшим за другий.
- Порівняння із значенням типу, який зберігається, здійснюється так, ніби з нього було створено std::optional.
- Те ж – для порівнянь із nullopt.

Інше

- Загортати bool в optional – не раціонально. По суті, отримується тернарне значення, типу boost::tribool. Детальніше див. аналіз тут: `https://www.boost.org/doc/libs/1_70_0/libs/optional/doc/html/boost_optional/tutorial/gotchas.html`.
- Аналогічно, optional<T*> ускладнює використання вказівника, не додаючи нічого – для вказівників існує спеціальне значення nullptr.
- `https://devblogs.microsoft.com/oldnewthing/20211004-00/?p=105754` – "Some lesser-known powers of std::optional", повчальна стаття.
- Обґрунтування, чому можна переміщати із об'єкту, збереженого в optional, без інвалідації самого optional: `https://akrzemi1.wordpress.com/2022/09/06/a-moved-from-optional/`.
- "The puzzle of trying to put an object into a std::optional", "Solving the puzzle of trying to put an object into a std::optional", "How do I put a non-copyable, non-movable, non-constructible object into a std::optional?".

C++23

- C++23 додає "monadic composition": and_then, or_else, transform та map.
- Приклад з стандарту:

```
std :: optional<image> get_cute_cat (const image& img) {
  return crop_to_cat(img)
    .and_then(add_bow_tie)
    .and_then(make_eyes_sparkle)
    .transform(make_smaller)
    .transform(add_rainbow);
}
```

де функції, передані add_then і т.д., має приймати і повертати std::optional, ймовірно – з різними збереженими типами.

- Детальніше див. https://www.fluentcpp.com/2019/07/16/expressiveness-nullable-types-and-composition-part-1/, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0798r3.html, https://www.fluentcpp.com/2019/07/19/expressiveness-nullable-types-and-composition-part-2/, https://www.cppstories.com/2023/monadic-optional-ops-cpp23/.

C++26

- C++23 додає std::optional<T&>.
  - Це такий собі optional, що не володіє об'єктом.
  - Ідейно схожий на variant<T&, nullptr_t>.
  - Оператор присвоєння перевизначає посилання.
  - Константність не глибока, для глибокої: std::optional<const T&>.
  - value_or() повертає T() за значенням.
- Також, optional зробили view (з одного або нуля елементів): "Give std::optional Range Support".
  - Для цього спеціалізує: `std::ranges::enable_view<std::optional<T>> == true`.
  - Також, додає iterator та const_iterator.

boost::optional

- Boost давно містить boost::optional, за яким, власне, і моделювався std::optional. Тому відмінності незначні, хоча й присутні.
- boost::optional не визначає спеціалізації std::hash, а для std::optional вона є.
- std::optional – literal type, і може бути використаним в constexpr-виразах, а boost::optional ні. (Поки?)
- std::nullopt -> boost::none.
- boost підтримує збереження в optional посилань, std – ні. Див. `https://www.boost.org/doc/libs/1_70_0/libs/optional/doc/html/boost_optional/tutorial/optional_references.html`. Увага, boost::optional<T&> не збільшують час життя того, на що посилаються! Тому їм не можна присвоювати посилання на тимчасові значення чи літерали (навіть для константних посилань).
- boost::optional вміють повертати вказівник на збережений об'єкт (метод get_ptr(), `https://www.boost.org/doc/libs/1_70_0/libs/optional/doc/html/boost_optional/reference/header__boost_optional_optional_hpp_/detailed_semantics___optional_values.html#reference_optional_get_ptr`).

- Метод boost::optional `value_or_eval()` повертає або вміст об'єкта, або результат виклику переданої йому функції. В епоху лямбда-функцій навряд чи потрібен.
- Метод boost::optional map() викликає вказану функцію для свого значення, якщо він не порожній та повертає optional із результатом (чи порожній). Приклад із документації:

```
auto length = [](const string& s){ return s.size(); };
optional<string> o1 {}, o2 {"cat"};
optional<size_t> os1 = o1.map(length), os2 = o2.map(length);
```

- Також, вони реалізовують багато функцій-не членів, типу:

```
auto get_optional_value_or ( optional<T>& o,
                             typename optional<T>::reference_type def )
        -> typename optional<T>::reference_type ;
```

Посилання

- https://en.cppreference.com/w/cpp/utility/optional.
- https://www.bfilipek.com/2018/05/using-optional.html – огляд використання std::optional.
- https://www.bfilipek.com/2018/04/refactoring-with-c17-stdoptional.html
- https://www.bfilipek.com/2018/05/errors-and-optional.html – std::optional не дуже пристосований для обробки, власне, помилок. Деякі міркування про це – в тексті. Також посилання звідти, про функціональний підхід до обробки помилок: https://blog.tartanllama.xyz/optional-expected/.
- https://www.boost.org/doc/libs/1_70_0/libs/optional/doc/html/boost_optional/tutorial/performance_considerations.html – міркування щодо продуктивності optional-типів, на прикладі boost::optional.
- https://www.boost.org/doc/libs/1_70_0/libs/optional/doc/html/index.html.
- "An option(al) to surprise you" – якщо optional містить вказівник, цей вказівник може бути нульовим.

std::variant – C++17

- std::variant – типобезпечне об'єднання.
- При оголошенні вказуються типи, які можуть в ньому зберігатися.
- Дозволяє звертатися лише до того значення, яке зараз реально в ньому присутнє.
- Ніколи не виділяє пам'ять динамічно[6].
- Ще один vocabulary type.
- Як і optional, походить із функціонального програмування. Відомий під іменами tagged union, discriminated union, sum type.
- C++20 додав можливість використання в constexpr-виразах.

[6] Звичайно, збережені в ньому об'єкти можуть це робити.

std::variant – використання

- std::variant – хороша альтернатива до union чи інших трюків, там де можна отримувати кілька різних типів – читаючи командний рядок, файл конфігурації тощо.
- Може використовуватися для обробки помилок (типу: variant<result, error_code>, але див. std::expected, доданого в C++23).
- Завдяки підтримці візиторів[7], можна використовувати для реалізації поліморфізму. Порівняння такого підходу із "звичним": `http://cpptruths.blogspot.com/2018/02/inheritance-vs-stdvariant-based.html`.
- Ще одним цікавим використанням є реалізації скінчених автоматів за їх допомогою – стани позначаються типами, а модифікація здійснюється візитором. Приклад: `https://www.bfilipek.com/2018/06/variant.html#state-machines` і посилання там. Кожен клас також може містити інформацію, пов'язану саме із цим станом – і плутанини не буде, див. `https://www.bfilipek.com/2019/06/fsm-variant-game.html` та оновлений варіант: `https://www.cppstories.com/2023/finite-state-machines-variant-cpp/`; `https://www.cppstories.com/2023/finite-state-machines-variant-vending-cpp/`.
- Поліморфізм із variant замість віртуальних функцій: `https://www.bfilipek.com/2020/04/variant-virtual-polymorphism.html`.
- Детальніше – див. також посилання далі.

[7] Тут правильніше казати "відвідувач" певне...

- Створення std::variant:

  ```
  std :: variant <int, double, std :: string > conf_item;
  ```

- За замовчуванням конструктором ініціалізується перший елемент, викликом його конструктора за замовчуванням.
- Можна ініціалізувати значеннями інших типів зі списку та присвоювати їх:

  ```
  conf_item = 3.14;
  std :: variant <int, double, std :: string > conf_item2{"none"s};
  ```

- Метод index() дозволяє взнати, елемент якого типу зберігається в variant зараз – повертає номер його типу у списку типів:

  ```
  std :: cout << "index = " << conf_item.index();
  ```

- Також, можна перевірити, чи є зараз в variant збережено даний елемент:

  ```
  if (std :: holds_alternative <std:: string >(conf_item)){ ..... }
  ```

- Звертання:

  ```
  std :: cout << "value = " << get<2>(conf_item) << std::endl;
  std :: cout << "value = " << get<std::string>(conf_item);
  ```

◇ Кине виключення `std::bad_variant_access`, якщо у variant зараз немає значення такого типу.

- Також, можна звертатися за допомогою методу get_if(), який повертає вказівник на елемент, якщо такий присутній, або нульовий вказівник, якщо ні:

```
if ( auto ptr = get_if<std::string>(&conf_item); ptr ) {     .....
if ( auto ptr = get_if<2>(&conf_item); ptr != nullptr ) {    .....
```

- Часто, більш зручним та більш ідіоматичним способом звертання є використання візиторів:

```
struct visiting_printer {
    void operator()(int i) const {
        std::cout << "Visitor int: " << i << std::endl;
    }
    void operator()(double d) const {
        std::cout << "Visitor float: " << d << std::endl;
    }
    void operator()(const std::string& s) const {
        std::cout << "Visitor string: " << s << std::endl;
    }
};

    .....
std:: visit ( visiting_printer {}, conf_item);
```

- Візитор повинен містити перевантаження для всіх варіантів.

Valueless by exception

- Оскільки динамічне виділення пам'яті заборонене[8], може виникнути ситуація, коли, через виключення конструктора, std::visitor загубив старе значення, але не набув нового.
- Перевірити це можна за допомогою методу `valueless_by_exception()`[9].
- При звертання до такого об'єкта за допомогою get() чи std::visit кине виключення `std::bad_variant_access`.
- Метод індекс у цьому випадку повертає `std::variant_npos`[10].

[8] За його допомогою можна було б реалізувати строгі гарантії щодо виключення, але така реалізація std::visitor була б дуже неефективною.

[9] `https://en.cppreference.com/w/cpp/utility/variant/valueless_by_exception`

[10] Гарантується, що це константа, рівна -1, але краще писати символьне ім'я!

Конструювання

- Елементи фундаментальних типів ініціалізуються нулем.
- Який елемент ініціалізувати під час конструювання визначається із типу аргументу.
- Якщо є неоднозначність, її можна вирішити за допомогою `std::in_place_index`:

```
// std :: variant <short, long> strange1{ 1 }; // Error !
std :: variant <short, long> strange2{ std :: in_place_index<1>, 1 };//long
```

- Цей же інструмент допомагає, коли конструюються складні об'єкти:

```
// variant <vector<int>, map<int, int>> strange3{{1, 2, 3}}; // Error
variant <vector<int>, map<int, int>> strange3{
                                    in_place_index<0>, {1, 2, 3}};
```

- Також можна скористатися `std::in_place_type`:

```
variant <vector<int>, map<int, int>> strange5{
                                    in_place_type<vector<int>>, {1, 2, 3}};
```

- Існують конструктори копіювання, переміщення та відповідні оператори присвоєння.

std::monostate

- Якщо перший елемент не може бути сконструйованим за замовчуванням, або його створення нераціональне, окрім переміщення його на інше місце в оголошенні – яке може бути небажаним, або й неможливим, коли всі елементи такі, можна скористатися std::monostate[11]:

```
struct TestStruct{
    explicit TestStruct(int){}
};
// std::variant<TestStruct, std::string> conf2; // Error!
std::variant<std::string, TestStruct> conf2a;
std::variant<std::monostate, TestStruct, std::string> conf2b;

std::variant<std::string, TestStruct> conf3{"abc"};
// std::variant<TestStruct, std::string> conf4{5}; // Error
std::variant<TestStruct, std::string> conf4a{TestStruct{5}};
```

- Це дуже простий клас, оголошений в std якось так:

```
struct monostate { };
```

Для нього визначено всі операції порівняння, всі екземпляри вважаються рівними між собою.

[11] "What's the point of std::monostate? You can't do anything with it!".

Модифікація

- Об'єкту типу std::variant можна присвоювати значення.
- get<>() теж дозволяє змінювати відповідний елемент:

```
get<int>(conf_item) += 3;
```

- Те ж можна робити, скориставшись вказівником, повернутим get_if():

```
if (auto ptr = get_if<int>(&conf_item); ptr )
    *ptr += 2;
```

- Метод emplace() дозволяє сконструювати нове значення на місці старого:

```
conf_item.emplace<std::string>("abc");
conf_item.emplace<2>("abc");
```

Для попереднього значення викликається деструктор.

- Візітор теж має можливість модифікувати:

```
struct visiting_modifier {
    void operator()(int& i) const {        i+=3;    }
            .....................................
};
std :: visit ( visiting_modifier {}, conf_item);
```

- Існує метод swap() та перевантажено відповідну функцію std::swap().

std::visit

- Вище std::visit використовувався лише для побічних ефектів. Однак, ця функція може повертати значення.
- Для цього всі перевантаження повинні повертати один і той же тип.
- Зручною є ідіома, коли візітор повертає std::variant того ж типу, що і отримав:

```
using variant_type_1 = std::variant<int, double, std::string>;
variant_type_1 conf_item3{"abc"s};
variant_type_1 w = std::visit ([]( auto&& arg) -> variant_type_1
                               {return arg + arg;}, conf_item3);
```

- std::visit може приймати декілька std::variant. Відповідний об'єкт-функція повинен реалізовувати перевантаження для всіх комбінацій типів:

```
struct visiting_poly_printer {
    void operator()(int i,    double d ) const {  .....  }
    void operator()(int i,    int j    ) const {  .....  }
    void operator()(double c, double d ) const {  .....  }
    void operator()(double d, int i    ) const {  .....  }
};
    ..............
    std::variant<int, double> v1{3}, v2{4.15};
    std::visit ( visiting_poly_printer {}, v1, v2);
```

- Оголошення окремих об'єктів-функцій доводиться робити окремо від виклику std::visit, навіть якщо вони дуже прості. Узагальнені лямбда-функції (generic lambda) підходять не завжди – вони виконують одну і ту ж операцію кожен раз, хай і з різними типами.
- Вирішити це можна за допомогою if constexpr[12]:

```
template<class T> struct always_false : std::false_type {};
        .....
std::visit([](auto&& arg) {
    using T = std::decay_t<decltype(arg)>; // Обдираємо від cv/ref.
    if constexpr (std::is_same_v<T, int>)
        std::cout << "int with value " << arg << '\n';
    else if constexpr (std::is_same_v<T, double>)
        std::cout << "double with value " << arg << '\n';
    else if constexpr (std::is_same_v<T, std::string>)
        std::cout << "std::string with value " << arg << '\n';
    else
        static_assert(always_false<T>::value, "Non-exhaustive visitor!");
}, w);
```

(Без always_false не працюватиме – потрібна залежність від T).

[12] Приклад із https://en.cppreference.com/w/cpp/utility/variant/visit.

std::visit

- C++20 надає ще таке перевантаження:

  template <**class** R, **class** Visitor , **class** ... Variants>
  constexpr R visit (Visitor &&, Variants&&...);

- C++23 додав підтримку std::visit для класів, успадкованих від std::variant.
- C++26 додав до цього класу метод visit.

std::visit з допоміжною структурою overloaded

- Використання if constexpr доволі громіздке. Запропонована в часи C++20 наступна конструкція в стандарт не потрапила, але залишилася в прикладах:

```
template<class... Ts> struct overloaded :
                  Ts ... { using Ts::operator ()...; };
template<class... Ts> overloaded(Ts ...) -> overloaded<Ts...>;
    ..............
```

```
std :: visit (overloaded {
      []( auto arg) { std :: cout << arg << ' '; },
      []( double arg) { std :: cout << std::fixed << arg << ' '; },
      []( const std :: string & arg) { std :: cout << arg << ' '; },
     }, w);
```

std::visit – боротьба із комбінаторним вибухом та default

- Використовуючи такий візитор для кількох variant, зручно скористатися auto для частини аргументів, якщо їх можна опрацьовувати одноманітно:

```
std :: visit (overloaded {
    ..........................
    [](type1, auto) { throw std::runtime_error("Wrong first type");
    ..........................          });
```

- А ось так можна зробити своєрідний default – щоб не реалізовувати всі комбінації:

```
[](auto, auto) {     .............     },
```

- Те ж можна зробити і з звичайними об'єкт-функціями, перевантаживши:

```
template <typename T1, typename T2>
auto operator()(T1, T2) { }
```

Подробиці std::variant

- Кілька елементів одного типу можуть бути присутні. Але це трохи безглуздо, крім того, перестане функціонувати присвоєння елементів відповідного типу, звертання за індексом і т.д.
- Перевантажено всі оператори порівняння. Якщо типи об'єктів, що порівнюються, однакові, якщо зберігаються однакові елементи – викликається їх оператор порівняння, якщо ж ні, вважається, що той, чий активний елемент має менший номер, є меншим.
- Про поведінку операторів порівняння за valueless_by_exception() == true (такий об'єкт вважається меншим за інші (?)) та при порівнянні різних типів std::variant<>, див. `https://en.cppreference.com/w/cpp/utility/variant/operator_cmp`.

- `std::variant_size` та `std::variant_size_v` дозволяють взнати розмір std::variant – кількість типів, що він може містити:

```
cout << "Types: " << variant_size<decltype(conf_item)>::value << endl
cout << "Types: " << variant_size_v<decltype(conf_item)> << endl;
```

Їх результати, природно, є константами часу компіляції.

- Типи відповідних елементів можна отримати за допомогою `std::variant_alternative` та `std::variant_alternative_t`:

```
std :: variant_alternative   <1, decltype(conf_item)>::type tx1;
std :: variant_alternative_t <1, decltype(conf_item)>       x2;
```

Це спеціалізований інструмент узагальненого програмування, зазвичай можна обійтися без нього – не зловживайте.

Ще раз про перетворення типів

- Згідно правил приведення типів C++, при виборі між const string reference та bool, C-стрічка, "Hello World" вирішить перетворюватися в bool. Це – неявне перетворення із C, а перетворення в стрічку – користувацьке, воно має менший пріоритет.
- Тому:

  ```
  std :: variant <std:: string , bool, int> var { 42 };
  var = "Hello World";
  ```

  вибере bool!
- Багфікс до стандарту, `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0608r3.html` це виправляє – надаючи перевагу перетворенням без урізання. (Так виходить, що в старому варіанті саме вони мають перевагу). Додано до C++20.
- Слайд створено за мотивами: `https://www.bfilipek.com/2019/07/surprising-conversions-char-bool.html`.

Пам'ять

- std::variant достатньо економний, хоча й привносить деякі накладні витрати на збереження поточного стану – значення якого ж типу збережено.
- Розмір, послідовно, type, variant<type>, variant<type, char>, variant<type, char, string>. GCC 8.3, MinGW; GCC 7.4, Linux; Clang 8.0, MinGW:

```
                  char  :  1 --  2 -- 2 -- 40
                   int  :  4 --  8 -- 8 -- 40
                 float  :  4 --  8 -- 8 -- 40
                double  :  8 -- 16 -- 16 -- 40
 std::complex<float>    :  8 -- 12 -- 12 -- 40
std::complex<double>    : 16 -- 24 -- 24 -- 40
    std::vector<int>    : 24 -- 32 -- 32 -- 40
std::map<int, string>   : 48 -- 56 -- 56 -- 56
```

- MSVC 2017 - x64:

```
                  char  :  1 --  2 -- 2 -- 48
                   int  :  4 --  8 -- 8 -- 48
                 float  :  4 --  8 -- 8 -- 48
                double  :  8 -- 16 -- 16 -- 48
 std::complex<float>    :  8 -- 12 -- 12 -- 48
std::complex<double>    : 16 -- 24 -- 24 -- 48
    std::vector<int>    : 32 -- 40 -- 40 -- 48
std::map<int, string>   : 24 -- 32 -- 32 -- 48
```

boost::variant

- Boost давно містить boost::variant, за яким, власне, і моделювався std::variant. Тому відмінності незначні, хоча й присутні.
- Boost::variant гарантує, що об'єкт завжди містить стан, такого поняття, як valueless_by_exception() не існує, але ціною можливості динамічного виділення пам'яті при ініціалізації. Детальніше див. `https://www.boost.org/doc/libs/1_70_0/doc/html/variant/design.html`.
- В boost аналог std::visit називається apply_visitor.
- boost::get не працює із індексами, лише з типами. Відповідно, немає підтримки дублювання типів. Є також boost::relaxed_get та boost::strict_get із тим же функціоналом, подробиці – див. `https://www.boost.org/doc/libs/1_70_0/doc/html/boost/relaxed_get.html` та `https://www.boost.org/doc/libs/1_70_0/doc/html/boost/get.html`. За проблем може кидати `boost::bad_get`.
- Замість std::monostate – boost::blank.
- Boost містить підтримку рекурсивних variant: `https://www.boost.org/doc/libs/1_70_0/doc/html/variant/tutorial.html#variant.tutorial.recursive.recursive-variant`.
- std::variant підтримує хешування, boost – ні.

QVariant

- QVariant із Qt ближчий до std::any, ніж до std::variant. Див. `https://doc.qt.io/qt-5/qvariant.html`.

Посилання

- https://www.bfilipek.com/2018/06/variant.html – детальна розповідь та аналіз можливих використань.
- https://arne-mertz.de/2018/05/modern-c-features-stdvariant-and-stdvisit/ – ще один, значно простіший огляд.
- https://www.bfilipek.com/2018/09/visit-variants.html – детальніше про візитори.
- https://en.cppreference.com/w/cpp/utility/variant/visit
- https://en.cppreference.com/w/cpp/utility/variant
- https://www.boost.org/doc/libs/1_70_0/doc/html/variant.html
- https://akrzemi1.wordpress.com/2016/02/27/another-polymorphism/ – про поліморфізм із використанням std::variant.
- http://cpptruths.blogspot.com/2018/02/inheritance-vs-stdvariant-based.html – зручна табличка-порівняння двох підходів до поліморфізму.
- http://khuttun.github.io/2017/02/04/implementing-state-machines-with-std-variant.html – приклад реалізації скінченого автомату на std::variant.
- https://www.cppstories.com/2018/06/variant/ – "Everything You Need to Know About std::variant from C++17", гарна оглядова стаття, містить оновлення по C++23.

- https://devblogs.microsoft.com/oldnewthing/20190620-00/?p=102604 – "Getting a value from a std::variant that matches the type fetched from another variant", трішки магії метапрограмування, для виклику функцій із парами однотипних елементів std::variant.

std::any – C++17

- std::any – більш загальний універсальний контейнер для значень, ніж std::variant.
- Може містити значення (практично) будь-якого типу.
- При чому, він – типобезпечний.
- Може потребувати динамічного виділення пам'яті, хоч старається того не робити.
- Код, що використовує std::any – дещо підозрілий, все ж C++ – строго типізована мова. Однак, він – значно краща альтернатива, ніж `void*`.
- std::any – не шаблон.
- Він зберігає типи, які є "decayed" – масиви перетворено у вказівники, cv-кваліфікатори та посилання забрано, за допомогою std::decay/std::decay_t.

Використання

- Бібліотечний код, який не знає, які саме дані прийдуть, наприклад, конектори SQL баз даних, код читання файлів із невідомим наперед списком типів тощо.
- Передача повідомлень між різними частинами програми, типу реалізації сигналів та слотів.
- Використання цього типу спрощує створення інтерпретаторів скриптових мов.
- Зазвичай, std::variant – краща альтернатива, оскільки список типів можна обмежити.

- Має конструктори за замовчуванням, може ініціалізуватися при створенні та присвоюватися:

```
std::any a1;
std::any a2{3};
a1 = 3.14;
a1 = "abc"s; // Семантично, те ж, що і a1 = std::string("abc");
```

- Перевірити, чи містить значення, можна за допомогою методу has_value():

```
cout << a1.has_value() << endl;
```

- Для звертання до збереженого значення слід використати any_cast. Якщо зараз збережено об'єкт не того типу, до якого відбувається перетворення – кине виключення std::bad_any_cast:

```
try {
    cout << "Int value: " << std::any_cast<int>(a1) << endl;
}catch(std::bad_any_cast& e){
    cout << "No int here" << endl;
}
```

Перетворення типів не здійснюються.

- Метод reset() знищує об'єт, збережений в std::any:

```
a1.reset();
```

- Аналогічно, при присвоєнні нового значення, для старого викликається деструктор.
- Метод type() дозволяє отримати об'єкт type_info для збереженого значення:

```
cout << a1.type().name() << endl;
```

- Якщо std::any не зберігає значення, type() повертає typeid(void).
- Ремарка, ці std::any зберігають значення різних типів:

```
a1 = "abc";
a1 = "abc"s;
```

Створення

- Крім ініціалізації об'єктами та конструкторів копіювання, підтримується також in-place ініціалізація, як із використанням `in_place_type`[13]:

```
std::any a3(std::in_place_type<std::complex<double>>, 3, 4);
std::any a4{std::in_place_type<std::string>, "ABC"};
```

  так і make_any():

```
auto a5 = std::make_any<std::string>("XYZ");
auto a6 = std::make_any<std::vector<int>>( {1, 2, 3} );
```

- Обидва цих способи запобігають створенню тимчасового об'єкта (хоча, сучасні компілятори їх і так не створюють зазвичай).

[13] "On naming things: The tension between naming something for what it is, what it does, or how it is used" – есей на тему таких назв.

Модифікація

- Існує оператор присвоєння.
- Метод reset() – очищає.
- Метод emplace() створює об'єкт нового типу, перед тим (викликом reset()) знищивши попередній:

  ```
  a6.emplace<std::vector<int>>( {1, 2, 3} );
  ```

- Якщо в процесі роботи emplace() сталося виключення – об'єкт не міститиме значення.
- Також реалізовано метод swap() та перевантажено відповідну функцію std::swap().

std::any_cast

- Вище std::any_cast використовувалася для отримання копії значення, збереженого в std::any.
- Вона також вміє повернути посилання чи вказівник на нього.

```
auto a5 = std::make_any<std::string>("XYZ");

any_cast<string&>(a5) += "Test";
cout << any_cast<const string&>(a5).size() << endl;
cout << any_cast<const string&>(a5) << endl;

auto a6 = std::make_any<std::vector<int>>( {1, 2, 3} );
    cout << any_cast<std::vector<int>>(&a6)->size() << endl;
```

- Зауважте, для отримання вказівника, слід std::any_cast передати вказівник на std::any – спроба перетворити до типу вказівника дасть, очікувано, std::bad_any_cast.
- Для посилання – кидатиме std::bad_any_cast, якщо перетворення неможливе.
- Для вказівника – повертає в цьому випадку nullptr.
-

Пам'ять

- Стандарт рекомендує використовувати Small Buffer Optimization, для уникнення зайвих виділень пам'яті.
- Для x86-64, GCC 8.3, MinGW; GCC 7.4, Linux; Clang 8.0, MinGW, sizeof(std::any) повернув 16.
- Для x86-32, GCC 8.3, MinGW, Clang 8.0, MinGW: 8
- MSVC 2017 x86-64: 64
- MSVC 2017 x86-32: 40

boost::any

- Boost давно містить boost::any, за яким, власне, і моделювався std::any. Тому відмінності незначні, хоча й присутні.
- boost не використовує Small Buffer Optimization.
- В boost::any немає emplace() та конструкторів із `in_place_type_t`.
- Обидва типи не хешуються.

- https://www.bfilipek.com/2018/06/any.html – детальна розповідь та аналіз можливих використань.
- https://en.cppreference.com/w/cpp/utility/any
- https://www.boost.org/doc/libs/1_70_0/doc/html/any.html

std::expected – C++23

- Спеціальний контейнер, який може зберігати або результат операції, або інформацію про помилку.
  - Альтернатива до виключень, більш зручний за ерзац-аналог з використанням std::variant, більш гнучкий у такій ролі, ніж std::optional, менш громіздкий та більш виразний, ніж коди помилок.
- Тривіальний приклад[14]:

```
std::expected<int, int> open_file(std::string_view name) {
  const int handler = open(name.data(), 0);

  if (handler == -1) {
    return std::unexpected{errno};
  }
  return handler;
}
      .....................
auto res = open_file("config.txt"sv);

if( !res.has_value()) {
  switch(res.error()) {
    case EACCES: std::cerr << "Permissions error" << std::endl;    break;
    case ENOENT: std::cerr << "File does not exist" << std::endl; break;
    default: std::terminate();
  } }
```

[14] Взято, із змінами, тут: https:

std::expected – C++23

- Звичайно, int в ролі коду помилки – дещо вироджений випадок. Варто мати спеціалізований тип для збереження інформації про неї.
- Шаблон оголошено так:

```
template< class T, class E >
class expected;
```

- Метод has_value() та відповідний оператор bool() дозволяють перевірити, чи є значення.
- value() повертає його, або кидає виключення std::bad_expected_access<E>, де збережено вміст об'єкту типу E, збереженого в std::expected.
- operator-> та operator* надають доступ до значення, або UB, якщо його немає.
- value_or() – або повернути значення із об'єкта, або, якщо його немає – значення за замовчуванням.
- error() – отримати об'єкт із помилкою. UB, якщо has_value() == true.
- Тип std::unexpected використовується для збереження об'єкту-помилки в std::expected.
- Є монадні операції, аналогічні відповідним з std::optional: and_then, transform, or_else, transform_error.
- Детальніше див.: `https://wg21.link/p2505`.
- Приклад використання: `https://www.cppstories.com/2024/expected-cpp23-monadic/`.

std::expected – посилання

- http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0323r3.pdf
- http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0323r7.html
- https://github.com/TartanLlama/expected
- https://isocpp.org/blog/2018/01/introduction-to-proposed-stdexpected
- https://foonathan.net/blog/2017/12/04/exceptions-vs-expected.html
- https://devblogs.microsoft.com/cppblog/cpp23s-optional-and-expected/
- ◇ Див. також: https://www.adit.io/posts/2013-04-17-functors,_applicatives,_and_monads_in_pictures.html – означення релевантних термінів.
- https://andreasfertig.blog/2022/09/cpp23-stdexpected-the-superior-way-of-returning-a-value-or-an-error/.
- https://mariusbancila.ro/blog/2022/08/17/using-the-cpp23-expected-type/.
- https://en.cppreference.com/w/cpp/header/expected.
- https://www.cppstories.com/2024/constexpr-number-parsing-cpp23/ – "Parsing Numbers At Compile Time with C++17, C++23, and C++26", зокрема – з використанням std::optional і std::expected.
- https://www.cppstories.com/2024/expected-cpp23-internals/ – есей про реалізацію.
- "Should std::expected be [[nodiscard]]?" – есей на тему.

std::string_view – C++17

- std::string_view – клас, що дозволяє доступатися до стрічки чи її частини, не володіючи нею.
- Служить лише для читання.
- При тому, він надає повноцінний інтерфейс роботи із стрічками, близький до інтерфейсу std::string.
- Зокрема, надає operator[], size() і т.д. стрічкові методи – див. далі.
- string_view може "вказувати" на частини стрічок інших видів – працює не тільки із std::string, але й із C-стрічками.
- Вирішують проблему "кожен framework має свою стрічку".
- Спрощує передачу стрічок між викликами.
- Дозволяє зменшити зусилля, щоб позбутися зайвих копіювань. `const string&` – зручний інтерфейс для цілої стрічки, але якщо в функцію слід передати лише її частину, він підходить не дуже – доводиться або робити копію частини стрічки, або передавати якісь індекси – псуючи інтерфейс функції-отримувача.
- Зокрема, substr(), що повертає string_view – значно ефективніший за той, що повертає копію.
- Технічно являє собою щось типу пари вказівника на початок "view" і розміру.

Interoperation

- Для передачі стрічок між різними бібліотеками "спільним знаменником" часто служить C-стрічка – const char* (і це добре, якщо ще const). Це проблемне рішення – слід або забороняти нульові байти (що заважає деяким кодуванням Unicode та іншим), отримуючи лінійну складність для визначення розміру, або окремо передавати розмір.
- const std::string reference – трохи краще рішення, але підходить не завжди. Воно справиться із std::string, C-стрічкою чи стрічковим літералом, але не зможе працювати із QString, wxString, стрічками MS COM, і т.д. і т.п.
- Можна вважати, що string_view – адаптер стрічок без володіння ними (non-owning).
- У більшості випадків може служити заміною `const std::string&`. Виняток – ситуація, коли функції користуються .c_str() – код, на щастя, після такої заміни перестане компілюватися, або коли використовуються .data(), неявно очікуючи присутності нульового символу – що буде UB, бо string_view не закінчується нульовим символом.

Приклад

```
void print_string(std::string_view s)
{
    std::cout << s << std::endl;
}
```

```
.............
const char text1[] = "abc";
std::string text2{"xyz"};
QString text3{"Qt!"};

print_string("Lit");
print_string(text1);
print_string(text2);
print_string(text3.toLatin1().constData());
print_string(text3.toLocal8Bit().constData());
```

- Жахіття із локалями ніде не ділися.
- Особливо гостро це відчувається, коли стрічкоподібних аргументів декілька:

```
bool compare(const std::string& s1, const std::string& s2);
```

- Особливо гостро це відчувається, коли стрічкоподібних аргументів декілька:

```
bool compare(const std::string & s1, const std::string & s2)
bool compare(const std::string & s1, const char* s2)
bool compare(const std::string & s1, const QString& s2)
bool compare(const char* s1, const std::string & s2)
bool compare(const char* s1, const char* s2)
bool compare(const char* s1, const QString& s2)
bool compare(const Qstring& s1, const std::string & s2)
bool compare(const Qstring& s1, const char* s2)
bool compare(const Qstring& s1, const QString& s2)
```

Так трохи зручніше:

```
bool compare(std::string_view s1, std::string_view s2);
```

- Класичним прикладом використання string_view є split():

```
vector<string_view> splitSV(string_view strv, string_view delims=" \n\t
    std :: vector<std::string_view> output;
    size_t first = 0;

    while ( first < strv.size()){
        const auto second = strv.find_first_of(delims, first );

        if ( first != second)
            output.emplace_back(strv.substr( first , second-first ));

        if (second == std::string_view::npos)
            break;

        first = second + 1;
    }
    return output;
}
```

- Код звідси: `https://www.bfilipek.com/2018/07/string-view-perf.html`.
- Взагалі, string_view добре товаришує із всіляким парсингом.

Небезпеки – 1

- string_view не зобов'язаний містити нульових символів (і не має методу .c_str(), але має метод .data(), що може ввести в оману).
- Зокрема, такий код має дуже серйозну загрозу UB!

```
string_view sv = ...;
string str1{sv.data()};  //! Дуже ймовірно, UB!
```

Слід або використати конструктор стрічки, що приймає string_view, або такий, що приймає крім вказівника на char, ще й розмір:

```
string str2 { sv };
string str3 { sv.data(), sv.size() };
```

- Також, жоден із засобів перетворення стрічок у числа, до появи в C++17 from_chars, не приймав інтервалу символів, а всі не-потокові (stringstream) засоби очікували нульовий символ в кінці стрічки. from_chars() цю проблему пом'якшили[15].

[15] Але не зняли повністю – через прагнення до максимальної ефективності, ігноруються локалі.

Небезпеки – 2

- Він не володіє відповідною стрічкою, тому може "провисати", як і звичайний вказівник чи посилання. Будьте обережні, зберігаючи в класі чи повертаючи із функції! Якщо string_view на локальний об'єкт ще може насторожити автора функції, то таке використання містить більш підступну помилку:

```
string_view do_nothing(string_view v){
    return v;
}

    auto sv2 = do_nothing(text2 + "ABC"); //! UB!
```

- Навіть так – вже UB:

```
std::string_view bad("a temporary string"s);
```

- А ось так – нормально, вказуватимете на статичний масив:

```
    auto sv1 = do_nothing("ABC");
    std::string_view good("a string literal");
```

string_view та map

- Завдяки heterogeneous lookup із C++14, можна робити так – шукати зразу за string_view, не створюючи проміжну стрічку:

```
map<string, int, less<>> words;
auto f = words.find(sv);
```

-

- Як і для std::string, std::string_view це `std::basic_string_view<char>`.
- Існують також варіанти wstring_view, u8string_view (C++20), u16string_view, u32string_view.
- Є контейнером:
  - містить всі необхідні typedef (value_type, iterator, const_iterator, size_type і т.д. і т.п.),
  - методи для отримання ітераторів (begin(), end(), rbegin(), cbegin(), rcbegin(), crbegin() та відповідні XXend()), C++20 (чомусь аж так пізно) додав перевантаження std::begin(), std::end().
  - методи size() та стрічковий length(), empty(), swap() і відповідне перевантаження std::swap(),
  - operator[], at(), front(), back(), data(),
  - існують всі оператори порівняння та перевантажено std::hash.
- Є стрічкою, хоч і константною, містить:
  - константу npos=-1, як індикацію кінця,
  - сімейство функцій пошуку: find, rfind, `find_first_of`, `find_last_of`, `find_first_not_of`, `find_last_not_of` із звичною поведінкою.
  - сімейство функцій compare(), аналогічні до string::compare.
  - перевантаження `operator<<`,
  - відповідний літерал operatorsv – "xyz"sv створить зразу string_view, без посередників.
- Звична функція для стрічок substr(), повертає string_view частини вихідної стрічки:

```
constexpr basic_string_view
    substr(size_type pos = 0, size_type count = npos ) const;
```

- copy() копіює вказану кількість літер у переданий масив символів. Копіює не більше, ніж їх всього є – безпечно передавати більші значення (як і для substr()):

  size_type copy(CharT* dest, size_type count, size_type pos = 0) **const**

  (constexpr в C++20).
- Цікаві функції – зменшення такого представлення зліва чи справа:

  constexpr **void** remove_prefix(size_type n);
  constexpr **void** remove_suffix(size_type n);

  $n <= size()$.
- C++20 додав ще два сімейства цікавих функцій – `starts_with()` та `ends_with()`, які дозволяють перевірити чи представлення починається або закінчується на задану підстрічку.
- C++23 додав метод contains()[16].
- C++26 дозволив конкатенацію стрічок та string_view.
- C++26 додав підтримку string_view під час створення stringstreams та bitset.

[16] Нарешті...

Посилання

- https://marcoarena.wordpress.com/tag/string_view/ – дуже гарний огляд.
- https://www.bfilipek.com/2018/07/string-view-perf.html, https://www.bfilipek.com/2018/07/string-view-perf-followup.html – прості тести продуктивності string_view.
- https://foonathan.net/blog/2017/03/22/string_view-temporary.html – про те, чому string_view приймає тимчасові значення, і чому це важливо. Якщо дуже коротко – мотив той же, чому const T& ініціалізується літералами та тимчасовими значеннями: для того, щоб всі const string& можна було на string_view замінити.
- http://www.modernescpp.com/index.php/c-17-avoid-copying-with-std-string-view – ще приклади зростання ефективності.
- https://en.cppreference.com/w/cpp/string/basic_string_view
- Аналогом string_view в boost є string_ref – див. https://www.boost.org/doc/libs/1_70_0/libs/utility/doc/html/string_ref.html. Див. також https://stackoverflow.com/questions/36704794/differences-between-booststring-ref-and-booststring-view.
- "How to convert between different types of counted-string string types" – цікава ілюстрація ролі string_view на прикладі стрічок, що бувають в API MS Windows.

std::span (C++20) (DRAFT)

- std::span – аналог std::string_view, який посилається на послідовність довільних елементів:

```
template< class T,
    std::size_t Extent = std::dynamic_extent
> class span;
```

- Extent – розмір.
- ◇ Використання dynamic_extent[17] вказує взяти цілий переданий масив.
- Там, де це має сенс – схожий на контейнер: front(), back(), operator[], at() (C++26), data(), size(), size_bytes(), empty(), отримання під-інтервалів: first(), last(), subspan(). Також, незалежні функції std::as_bytes() та std::as_writable_bytes().
- Підтримує запис у елементи.

[17] Дорівнює максимальному значенню size_t.

- Детальніше див. на cppreference. Приклад звідти, з доповненнями:

```
template<class T, std::size_t N, std::size_t M>
constexpr bool contains(std::span<T,N> span, std::span<T,M> sub) {
    return std::search(span.begin(), span.end(), sub.begin(),
                       sub.end()) != span.end();
}
    ..................
    constexpr int a[] {0, 1, 2, 3, 4, 5, 6, 7, 8};
    constexpr int b[] {8, 7, 6};
    std::vector<int> c = { 11,12,13,14,15 };
    bool r1 = contains(std::span{a}, std::span{a + 1, 4});
    bool r2 = contains(std::span{a, 8}, std::span{a, 9});
    bool r3 = contains(std::spanc{}, std::span{a, 3});
```

- Приклади: "How to use std::span from C++20".
- Огляд схожих конструкцій C++: "Spans, string_view, and Ranges - Four View types (C++17 to C++23)".
- В C++26 додали span::at() та можливість конструювати з std::initializer_list – часто корисний засіб.

std::mdspan (C++23) (DRAFT)

- std::mdspan – варіант span, який інтерпретує інтервал як багатомірний:

```
template< class T,
      class Extents,
      class LayoutPolicy = std::layout_right,
      class AccessorPolicy = std::default_accessor<T>
> class mdspan;
```

- Extents – типу std::extents, вказує кількість і величину вимірів.
- LayoutPolicy визначає, як цей багатомірний масив відображається на одномірний. Наприклад, з допомогою нього можна працювати з масивами FORTRAN.
- AccessorPolicy вказує, як перетворити одномірний індекс в посилання на елемент.
- Приклад з cppreference:

```
std::vector v{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};

// View data as contiguous memory representing 2 rows of 6 ints each
auto ms2 = std::mdspan(v.data(), 2, 6);
// write data using 2D view
for (std::size_t i = 0; i != ms2.extent(0); i++)
    for (std::size_t j = 0; j != ms2.extent(1); j++)
        ms2[i, j] = i * 1000 + j;
```

std::mdspan (C++23) – посилання

- Детальніше див. цю відповідь на запитання "What is an mdspan, and what is it used for?" та "A Gentle Introduction to mdspan".
- "Adjacency Matrix and std::mdspan, C++23" – приклад використання.
- "Details of std::mdspan from C++23" – есей з прикладами на тему.
- В C++26 додали конструювання з константами часу компіляції:

```
int arr[5] = {1, 2, 3, 4, 5};
int* p = arr;
// Було:
mdspan ms (p, 3); // mdspan<int, extents<size_t, dynamic_extent>>
mdspan ms2(p, integral_constant<size_t, 3>{}); // same
// Стало:
mdspan ms3(p, extents<size_t, 3>); //  mdspan<int, extents<size_t, 3>>
```

std::submdspan – C++26

- В C++26 запропоновано std::submdspan (з <mdspan>) – клас для створення одномірних зрізів багатомірних масивів.
- Наприклад, якщо є двомірний масив RGB-кольорів, так можна звернутися до червоного кольору одного рядка:

```
enum class Colors: unsigned { red, green, blue, last };
constexpr auto n_colors = static_cast<unsigned>(Colors::last);
constexpr unsigned rows = 320, cols = 640;

std::byte screen[rows][cols][n_colors] = {};

auto red_row = std::submdspan(screen, 1, std::full_extent,
                              static_cast<unsigned>(Colors::red));
  for(size_t col = 0; col != red_row.extent(0); ++col)
  { ..... }
```

- Детальніше див. пропозицію: "Submdspan".
- На жаль, станом на березень 2024, не підтримується компіляторами.

Розумні вказівники

- Керування пам'яттю в C++ складне і є джерелом помилок.
- Воно сильно спрощується, якщо всі ресурси, які виділяються, загортати у відповідні класи, деструктори яких звільнятимуть виділені ресурси.
- Такий підхід називається "Resource Acquisition Is Initialization"[18], або RAII — "Отримання ресурсу є ініціалізація".
- Однак, писати такий клас кожен раз, коли потрібно динамічно виділити щось, незручно. Та й, в загальному випадку, це нетривіальна задача.
- Для спрощення роботи у таких випадках було винайдено поняття розумного вказівника — smart pointer.
- Розумний вказівник (smart pointer)[19] — сутність, яка поводиться як вказівник[20], але при тому виконує додаткові операції – звільнення пам'яті при виході з області видимості, перевірка виходу за межі, тощо.

---

[18] Див. `http://uk.wikipedia.org/wiki/Resource_Acquisition_Is_Initialization` та англійську версію цієї ж статті. Див. також есей про RAII в C: "Why Not Just Do Simple C++ RAII in C?".

[19] `https://en.wikipedia.org/wiki/Smart_pointer`.

[20] Зокрема, для неї перевизначено оператори розіменування та звертання до поля за вказівником на клас чи структуру.

std::unique_ptr

- Стандартні розумні вказівники C++ оголошено в <memory>.
- std::unique_ptr – розумний вказівник, що володіє об'єктом.
- Його не можна копіювати, але **можна переміщати**.
- ◇ Завдяки цьому його можна повертати з функцій чи передавати функціям із переміщенням.
- Існує два варіанти – для одного елемента і для масиву:

```
template< class T, class Deleter = std::default_delete<T> >
class unique_ptr;
template< class T, class Deleter >
class unique_ptr<T[], Deleter>;
```

- Приклад:

```
{
    std::unique_ptr<int> isp1{new int{42}};
        .....
    *isp1 = 13;
        .....
} // Тут обєкт' буде автоматично знищено, викликом delete.
```

- Більш ідіоматично створювати так:

```
auto isp2 = std::make_unique<int>(42);
```

- Використання для масивів – де видалення буде відбуватися за допомогою delete[]:

```
std::unique_ptr<int[]> iasp1{new int[5]};
auto iasp12 = std::make_unique<int[]>(5);
```

◇ Зауважте – тепер аргументом make_unique замість аргументів для конструктора, передається розмір масиву.

- Видалення відбувається:

◇ В момент знищення розумного вказівника.

◇ Коли йому присвоюється інше значення оператором =.

◇ Коли викликається його метод reset(), який або присвоює нове значення, або nullptr.

- На додачу до operator* та operator->, перевантажено operator[] та operator bool із очікуваною семантикою.
- Також, є всі оператори порівняння.
- В C++20 додали operator<<.
- Може керувати будь-якими вказівникоподібними об'єктами, які є NullablePointer – тобто, їх можна порівнювати з std::nullptr_t.

- Також, можна передати свій код, який потрібно викликати для знищення об'єкту:

```
void close_file (std::FILE* fp){
    std::fclose(fp);
}
    ....................
    std::unique_ptr<std::FILE, decltype(&close_file)>
        fileh1{std::fopen("../CMakeLists.txt", "r"), &close_file};
```

  - Під час виходу змінної fileh з області видимості, файл буде закрито викликом close_file.
  - Працювати з тим, щоправда, доведеться якось так:

```
size_t ret_code = std::fread(arr, 1, arr_z1, &*fileh1);
```

- Інші способи передати свій delete:

```
std::unique_ptr<std::FILE, void(*)(std::FILE*)>
                    fileh2{std::fopen("../CMakeLists.txt", "r"),
                    [](auto fp){ std::fclose(fp);} };

std::unique_ptr<std::FILE, decltype(std::fclose)*>
                    fileh3{std::fopen("../CMakeLists.txt", "r"),
                    std::fclose };
```

- "RAII all the things?" – приклад, як загорнути в unique_ptr файлові дескриптори POSIX – які не є вказівниками, тому потрібно використати клас-обгортку:

```
class file_descriptor {
    int fd_{-1};
public:
    file_descriptor(int fd = -1): fd_(fd) {}
    file_descriptor(nullptr_t) {}
    operator int() const { return fd_; }
    explicit operator bool() const { return fd_ != -1; }
    friend bool operator==(file_descriptor, file_descriptor) = default;
}; // Всі функції можна помітити як noexcept та constexpr

struct fd_closer {
    using pointer = file_descriptor; // Магія тут
    void operator()(pointer fd) const noexcept {
        ::close(int(fd));
    }
};

std::unique_ptr<file_descriptor, fd_closer> t = ...;
```

Працює це завдяки синоніму типу pointer.

"Протягування" unique_ptr

- Копіювати unique_ptr не можна, але можна переміщати.
- ◇ Завдяки цьому його можна передавати та отримувати з функцій:

```
std::unique_ptr<int> pass_through(std::unique_ptr<int> arg) {
    return arg;
}

.....

std::unique_ptr<int> isp3{new int{42}};
std::unique_ptr<int> isp4 = pass_through(std::move(isp3));
```

std::shared_ptr

- shared_ptr – розумний вказівник, який можна копіювати:

  template< **class** T > **class** shared_ptr;

- Він підтримує лічильник копій та знищує об'єкт, коли кількість копій самого розумного вказівника стає рівною нулю.
- Копіювати можна сам shared_ptr – якщо ініціалізувати два і більше shared_ptr тим самим "сирим" вказівником – це UB.
- Оскільки мусить підтримувати контрольний блок пам'яті із лічильником та іншими службовими даними[21], роль std::make_shared[22] помітно більша, ніж std::make_unique – вона підвищує ефективність, алокуючи пам'ять для керованого об'єкту і для лічильника shared_ptr разом.
- В цілому схожий за інтерфейсом на unique_ptr.
- Важливий додатковий метод: use_count().
- Не варто зловживати std::shared_ptr – часто достатно std::unique_ptr.

[21] Вказівник на deleter, алокатор, лічильник shared_ptr об'єктів, що посилаються на нього, лічильник weak_ptr, що посилаються на нього, вказівник на об'єкт, яким керує – або сам цей об'єкт.

[22] Існує також std::allocate_shared, яка використовує переданий алокатор.

Потокобезпечність

- Використання різних копій shared_ptr, що посилаються на той самий контрольний блок із різних потоків потокобезпечне. Звертання до одного об'єкта shared_ptr з різних потоків – ні.
- Щодо всіляких хитрих способів атомарно використовувати один і той самий об'єкт, див: `https://en.cppreference.com/w/cpp/memory/shared_ptr/atomic`, однак, майте на увазі: "*To avoid data races, once a shared pointer is passed to any of these functions, it cannot be accessed non-atomically. In particular, you cannot dereference such a shared_ptr without first atomically loading it into another shared_ptr object, and then dereferencing through the second object.*".

- Делетер передається в конструкторі[23], але щоб вказати використання масиву, потрібно передати відповідний аргумент шаблону (з C++17 – до того не вмів працювати з масивами, вирішувалося користувацьким делетером).
- ◇ std::make_shared[24] не дозволяє передавати делетер.
- Може конструюватися з unique_ptr (з переміщенням) та weak_ptr.
- Може керувати одним вказівником, але зберігати інший. Наприклад, може містити вказівник на підоб'єкт об'єкту, вказівник на який буде використано під час видалення:

  template< **class** Y >
  shared_ptr(**const** shared_ptr<Y>& r, element_type* ptr) noexcept;

- ◇ Видалятиме він той об'єкт, на який вказує r, але розіменування, get() тощо працюватимуть з ptr.
- Детальніше див. `https://en.cppreference.com/w/cpp/memory/shared_ptr/shared_ptr`.

[23] Щодо можливих причин – див. `https://leimao.github.io/blog/CPP-Smart-Pointer-Custom-Deleter/`.

[24] `https://en.cppreference.com/w/cpp/memory/shared_ptr/make_shared`.

std::weak_ptr

- Кожна копія std::shared_ptr продовжує час існування того, на що вона посилається, на час свого існування.
- Це не завжди зручно.
  - Наприклад, якщо в грі "Астероїди", кожен астероїд має вказівники на своїх сусідів, наявність таких вказівників не повинна заважати знищувати конкретні астероїди.
  - Якщо всі такі вказівники зробити shared_ptr, отримаємо циклічну залежність — вказівники на конкретний астероїд завжди існуватимуть (в його сусідах, як мінімум), деструктор викликати неможливо. Скористатися звичайними вказівниками теж не вдасться.
- Якраз на такий випадок існують "слабкі" вказівники.
- weak_ptr зберігає посилання на контрольний блок std::shared_ptr, однак не впливає на час його життя.
- Безпосередньо розіменовувати його не можна — він може вказувати на об'єкт, якого не існує.
- Однак, його можна перетворити на std::shared_ptr.
- Якщо отриманий розумний вказівник не буде нульовим, значить об'єкт ще існує, а тепер продовжить існувати, поки створений std::shared_ptr не буде знищено.

- Конструюється з shared_ptr.
- Метод lock() перетворює в shared_ptr.
- Приклад з `https://en.cppreference.com/w/cpp/memory/weak_ptr/weak_ptr`:

```
struct Foo {};

    .....
std::weak_ptr<Foo> w_ptr;

{
    auto ptr = std::make_shared<Foo>();
    w_ptr = ptr;
    std::cout << w_ptr.use_count() << '\n';
}

std::cout << w_ptr.use_count() << '\n';
std::cout << std::boolalpha << w_ptr.expired() << '\n';
}
```

std::enable_shared_from_this

- Публічне успадкування від std::enable_shared_from_this дозволяє створювати нові копії shared_ptr для цього об'єкту, які користуються тим же контрольним блоком.
- Отримання такої копії здійснюється викликом shared_from_this(), який додається після такого успадкування.
- C++17 додала також weak_from_this().
- Викликати можна, лише якщо об'єкт вже керується shared_ptr – інакше це UB до C++17, або – виключення std::bad_weak_ptr після.
- Детальніше див. `https://en.cppreference.com/w/cpp/memory/enable_shared_from_this`.
- `https://devblogs.microsoft.com/oldnewthing/20220720-00/?p=106877` – "My class derives from std::enable_shared_from_this, but shared_from_this() doesn't work"

std::enable_shared_from_this

- Приклад:

```
class Best : public std::enable_shared_from_this<Best> {
public:
    std::shared_ptr<Best> getptr() {
        return shared_from_this();
    }
    // No public constructor, only a factory function
    [[nodiscard]] static std::shared_ptr<Best> create() {
        // Not using std::make_shared<Best> because the c'tor is private.
        return std::shared_ptr<Best>(new Best());
    }
private:
    Best() = default;
};

    std::shared_ptr<Best> best0 = Best::create();
    std::shared_ptr<Best> best1 = best0->getptr();
```

- Взято із: `https://en.cppreference.com/w/cpp/memory/enable_shared_from_this`.

C++20

- C++20 додав нові функції створення, що не ініціалізують виділені масиви:

  std::make_unique_for_overwrite()
  std::make_shared_for_overwrite()
  std::allocate_shared_for_overwrite()

- Приклад:

  ```
  std::unique_ptr<int[]> arr = std::make_unique_for_overwrite<int[]>(size);
  ```

- Детальніше див.
  `https://en.cppreference.com/w/cpp/memory/unique_ptr/make_unique`.

std::out_ptr() та std::inout_ptr() – C++23

- C++23 додав функції std::out_ptr() та std::inout_ptr() для покращення взаємодії розумних вказівників і C API.
- Вони дозволяють більш лаконічно керувати вказівником, створеним функцією C, за допомогою розумних вказівників:

```
std::unique_ptr<sqlite3, decltype(close_db)> up;
sqlite3_open(":memory:", std::out_ptr(up));
```

- Їх результат можна передавати функціям C, що отримують вказівники.
- Ці функції повертають об'єкти типу std::out_ptr_t та std::inout_ptr_t.

- std::inout_ptr_t потрібно використовувати з функціями, які оновлюють вказівник (часто – отримавши його як вказівник на вказівник):

```
int  foreign_resetter(T**);
std::unique_ptr<T, D> up;

foreign_resetter(std::inout_ptr(up))
```

робить щось таке:

```
T *raw_p = up.get();
up.release();
foreign_resetter(&raw_p);
up.reset(raw_p);
```

- std::out_ptr_t – з тими, що лише його перезаписують:

```
foreign_setter(std::out_ptr(up))
```

робить щось таке:

```
T* raw_p{};
foreign_setter(&raw_p);
up.reset(raw_p);
```

- Приклади взято з cppreference: std::inout_ptr_t, std::out_ptr_t, std::out_ptr()

"Боротьба" з надміру розумними вказівниками

- std::addressof() з C++11 дозволяє отримати справжню адресу об'єкта, навіть якщо перевантажено operator&.
- std::to_address() з C++20 дозволяє з так званих Fancy pointers[25], отримати "звичайний" вказівник. Для звичайних вказівників просто повертає їх.
  - ◇ На відміну від std::addressof() може застосовуватися до не ініціалізованої пам'яті.

[25] https://en.cppreference.com/w/cpp/named_req/Allocator#Fancy_pointers.

C++26 – std::indirect та std::polymorphic

- std::indirect – клас для об'єктів, які виділяються динамічно, але мають поводитися, як значення.
  - ◇ Зокрема, копіює об'єкт, на який посилається під час свого копіювання.
  - ◇ Не null, крім як коли з нього перемістили (і тоді метод valueless_after_move() поверне true).
- "The PImpl idiom and the C++26 std::indirect type" – приклад використання.
- Доступ до самого об'єкта – як для звичайного вказівника:
- std::polymorphic – його поліморфний варіант, може містити об'єкт чи його нащадки.
- "P3019: indirect and polymorphic: Vocabulary Types for Composite Class Design".
- Приклад std::indirect на наступному слайді, std::polymorphic – на godbolt.

```
class Counter {
    struct Impl;                std :: indirect <Impl> impl_;
public:
    Counter() = default;
    ~Counter() = default;
    Counter(const Counter&) = default;
    Counter& operator=(const Counter&) = default;
    Counter(Counter&&) noexcept = default;
    Counter& operator=(Counter&&) noexcept = default;

    void set(int value);
    int get() const;
};

// ==================== .cpp файл
struct Counter::Impl {
    int value = 0;
};

void Counter::set(int value) {
    impl_->value = value;
}
int Counter::get() const {
    return impl_->value;
}
```

boost smart pointers

- Реалізація розумних вказівників в стандарті відпрацьовувалася в boost: `https://www.boost.org/doc/libs/1_82_0/libs/smart_ptr/doc/html/smart_ptr.html`.
- Ввійшли в стандарт не всі.
- Наприклад, boost::intrusive_ptr – на відміну від shared_ptr, він передає керування обліком самому об'єкту, викликаючи функції intrusive_ptr_add_ref() та intrusive_ptr_release().
  - Він зручний, якщо об'єкт вже має якийсь підрахунок посилань. Також, він може бути ефективнішим за shared_ptr.
- boost::local_shared_ptr – варіант shared_ptr, який не є потокобезпечним. В однопоточному застосуванні він ефективніший.

Інші розумні вказівники

- Qt має великий набір власних розумних вказівників[26].
- Поміж них – QPointer. Цей вказівник встановлюється в nullptr коли об'єкт, на який він вказує, знищується. Може вказувати лише на нащадків QObject.
- Hazard pointers[27], які використовуються при створенні lock-free структур даних, теж можна вважати різновидом розумних вказівників.
- std::auto_ptr – застарілий розумний вказівник із C++98, невдала спроба створити розумний вказівник – коли його копіювати: "ptr1 = ptr2", ptr2 стає нульовим вказівником.
- Ітератори, зазвичай, розумними вказівниками не називають.
- `https://github.com/zhuowei/nft_ptr` – кумедний розумний вказівник, представляє об'єкти як NFT-токени Ethereum блокчейну.
- Ще цікавий приклад: `https://quuxplusone.github.io/blog/2018/08/11/the-auto-macro/`, `https://quuxplusone.github.io/blog/2024/02/14/auto-macro-feature-requests/`.

[26] Вони там з'явилися раніше, ніж в стандарті C++.

[27] `https://en.wikipedia.org/wiki/Hazard_pointer`.

Нестандартні розширення

- GCC дозволяє робити щось таке:

```
void example_usage() {
    __attribute__((cleanup(fclosep))) FILE * logfile
                    = fopen(" logfile . txt ", "w+");
    fputs(" hello   logfile !",   logfile );
}
```

Приклад реалізації

- Приклад взято із книги Nicolai M. Josuttis: "The C++ Standard Library": `http://www.josuttis.com/libbook/cont/countptr.hpp.html`.

```
template <class T>
class CountedPtr {
  private:
    T* ptr;          // pointer to the value
    long* count;     // shared number of owners

  public:
    // initialize pointer with existing pointer, obtained by new
    explicit CountedPtr (T* p=0)
     : ptr(p), count(new long(1)) {
    }

    // copy pointer (one more owner)
    CountedPtr (const CountedPtr<T>& p) throw()
     : ptr(p.ptr), count(p.count) {
        ++*count;
    }

    // destructor (delete value if this was the last owner)
    ~CountedPtr () throw() {
        dispose ();
    }
```

```
    // assignment (unshare old and share new value)
    CountedPtr<T>& operator= (const CountedPtr<T>& p) throw() {
        if (this != &p) {
            dispose();
            ptr = p.ptr;
            count = p.count;
            ++*count;
        }
        return *this;
    }

    // access the value to which the pointer refers
    T& operator*() const throw() {
        return *ptr;
    }
    T* operator->() const throw() {
        return ptr;
    }

private:
    void dispose() {
        if (--*count == 0) {
            delete count;
            delete ptr;
        }
    }
```

Посилання

- Огляд розумних вказівників на en:wiki, див. також український варіант статті.
- "Smart Pointers - What, Why, Which?" і "Smart Pointers Overview" — більш детальній, але доступніший, огляди питання.
- Приклад реалізації розумних вказівників від Джосаттіса".
- "Smart Pointers" – детальніше про їх реалізацію, за мотивами славнозвісної книги Александреску "Modern C++ Design".
- Розумні вказівника boost.
- QPointer, QScopedPointer, QScopedArrayPointer, QSharedPointer, QWeakPointer, QSharedDataPointer – розумні вказівники Qt.
- Розумні вказівники C++ на CppReference.
- "Brief tutorial on unique_ptr", 2011.
- "Thoughts on creating a tracking pointer class, part 1: Concept art", "Part 2: Using a std::list", "Part 3: Using a std::vector", "Part 4: Using a circular doubly linked list", "Part 5: Copying our tracking pointer", "Part 6: Non-modifying trackers", "Part 7: Non-modifying trackers, second try", "Part 8: Tracking const objects", "Part 9: Conversion", "Part 10: Proper conversion", "Part 11: Repairing assignment", "Part 12: A shared tracking pointer", "Part 13: Restoring the strong exception guarantee", "Part 14: Nonthrowing moves with the shared tracking pointer", "Part 15: A custom shared pointer", "Part 16: Second attempt to use a list".
- "Using RAII to remedy a defect where not all code paths performed required exit actions" та "Using RAII to remedy a defect where not all code paths performed required exit actions, follow-up".

- "Why is there a make_unique? Why not just overload the unique_ptr constructor?", деякі міркування про потребу в make_unique.
- "Why do smart pointers null out the wrapped pointer before destroying it?".
- "Wrapping Resource Handles in Smart Pointers".
- "Making sure that people use make_unique and make_shared to make your object".
- "Inside STL: Smart pointers".
  - "Inside STL: The shared_ptr constructor vs make_shared".
  - "What it means when you convert between different shared_ptrs".
  - "Phantom and indulgent shared pointers", про розумні вказівники, які або вказують на nullptr, або не володіють об'єктом (приклад – розумний вказівник на глобальний об'єкт).

std::format з <format>

- Інструмент гнучко форматувати стрічки, аналогічна C-printf, форматуванню стрічок в Python, Qt QString::arg() тощо.

```
auto s1 = std::format("x = {}, y = {}, z = {}\n",
                      1, 2.34, std::chrono::days(42));
auto s2 = std::format("a = {:5.3f}, b = {:#x}", 3.14, 0xAB);
```

- Однак, використання з не-константними стрічками форматування утруднене – vformat приймає динамічні стрічки формату, однак, аргументи мають бути "запакованими" в тип std::format_args:

```
return std::vformat(runtime_format, std::make_format_args(arg1, arg2));
```

- Хоча це суб'єктивно, але використовувати fmtlib, на базі якої відбувалася стандартизація – помітно зручніше, не зважаючи на вагомі аргументи на користь constexpr-стрічок формату.
- В C++26 додали трохи зручніший варіант:

```
return std::vformat(std::runtime_format(runtime_format), arg1, arg2);
```

- В цілому, може призводити до роздування коду, але, зазвичай, багато швидша за iostream.
- `https://en.cppreference.com/w/cpp/utility/format/format` – подробиці.

- std::format_to() виводить результат у переданий ітератор для виводу:

```
std :: string   buffer ;

std :: format_to(
    std :: back_inserter( buffer ),
    "Hello ,  C++{}!\n",
    "20");
```

- Приклад з: `https://en.cppreference.com/w/cpp/utility/format/format_to`.
- std::vformat_to() – аналогічно до std::vformat(), приймає динамічну стрічку формату, і теж відносно багатослівна.
- std::format_to_n() виводить не більше переданої їй кількості символів. Однак, vformat_to_n – відсутня.
- Функція std::print() з <print> – отримує ті ж аргументи, що і std::format, і виводить їх на консоль, в потік або у std::FILE.
- Існує також std::println().

C++26

- C++26 додав підтримку форматування filesystem::path:

```
fs::path p = "/usr/local/bin/clang++";
std::println("{}",   p);    // /usr/local/bin/clang++
std::println("{:?}", p);    // "/usr/local/bin/clang++"
fs::path p = R"(C:\Users\marius\file.txt)";
std::println("{}",     w);  // C:\Users\marius\file.txt     (native separators)
std::println("{:g}",   w);  // C:/Users/marius/file.txt     (generic)
std::println("{:g?}",  w);  // "C:/Users/marius/file.txt"   (generic + escaped)
```

  Приклад взято тут: "Improvements to std::format in C++26"

- Додано підтримку форматування вказівників:

```
void* p = ....;
std::println("{}",     p);
std::println("{:P}",   p);
```

- Можливість виклику std::println() без аргументу.
- C++26 Runtime format strings II додав std::runtime_format(str) – нарешті: "std::format(std::runtime_format(str), 42);".
- Додано також підтримку використанню в constexpr: "constexpr std::format".

- Написання форматерів для власних класів: "std::format from scratch, part 1", "part 2", "part 3".
- "Formatting Custom types with std::format from C++20"
- Деякі нюанси та порівняння із fmtlib: "Experience with std::format".
- "Honey, I shrunk {fmt}: bringing binary size to 14k and ditching the C++ runtime" – про те, як мінімізувати виконавчий файл. Згадує цікавий інструмент – Bloaty.
- "C++26: std::format improvement (Part 1)", "Part 2", "Format your own type (Part 1)", "Part 2".

std::experimental

- Деякі інструменти, описані далі, (як і багато інших), перш ніж потрапити в std::, якийсь час живуть в в просторі імен std::experimental.
- Якщо ваш компілятор старіший – замість, скажімо, <optional>, вам може довестися включати <experimental/optional> (`https://en.cppreference.com/w/cpp/experimental/optional`).
- Див. також `https://en.cppreference.com/w/cpp/experimental/lib_extensions` та `https://en.cppreference.com/w/cpp/experimental`.

Floating point-константи – C++20

- Нарешті, було додано такі речі, як число пі...

```
namespace std::numbers {
  template<class T> inline constexpr T e_v           = /* unspecified */;
  template<class T> inline constexpr T log2e_v       = /* unspecified */;
  template<class T> inline constexpr T log10e_v      = /* unspecified */;
  template<class T> inline constexpr T pi_v          = /* unspecified */;
  template<class T> inline constexpr T inv_pi_v      = /* unspecified */;
  template<class T> inline constexpr T inv_sqrtpi_v = /* unspecified */;
  template<class T> inline constexpr T ln2_v         = /* unspecified */;
  template<class T> inline constexpr T ln10_v        = /* unspecified */;
  template<class T> inline constexpr T sqrt2_v       = /* unspecified */;
  template<class T> inline constexpr T sqrt3_v       = /* unspecified */;
  template<class T> inline constexpr T inv_sqrt3_v   = /* unspecified */;
  template<class T> inline constexpr T egamma_v      = /* unspecified */;
  template<class T> inline constexpr T phi_v         = /* unspecified */;
  // Та всі відповідні:
  template<floating_point T> inline constexpr T e_v<T> = ...;
   inline  constexpr double e                  = e_v<double>;
}
```

Безпечні математичні функції – C+20

- Сімейство функцій виду (`https://en.cppreference.com/w/cpp/numeric/lerp`):

  constexpr **double** lerp(**double** a, **double** b, **double** t) noexcept;

  Розраховують лінійну інтерполяцію між a та b (для $|t| < 1$), або лінійну екстраполяцію для інших значень t.
- Пошук середньої точки, без переповнення для цілих типів:

  template< **class** T >
  constexpr T midpoint(T a, T b) noexcept;
  template< **class** T >
  constexpr T* midpoint(T* a, T* b);

◇ "Generalizing std::midpoint" – есей на тему.

Безпечні порівняння – C++20

- Порівнювати знакові та беззнакові типи в C та C++ небезпечно – спершу відбувається перетворення знакового до беззнакового, тому результат для від'ємних буде невірним.
- Тому до <utility> було додано функції безпечного такого порівняння:

```
template <class T, class U>
constexpr bool cmp_equal (T t , U u) noexcept
template <class T, class U>
constexpr bool cmp_not_equal (T t , U u) noexcept
template <class T, class U>
constexpr bool cmp_less (T t , U u) noexcept
template <class T, class U>
constexpr bool cmp_greater (T t , U u) noexcept
template <class T, class U>
constexpr bool cmp_less_equal (T t , U u) noexcept
template <class T, class U>
constexpr bool cmp_greater_equal (T t , U u) noexcept
template <class R, class T>
constexpr bool in_range (T t) noexcept
```

- Детальніше див. https://en.cppreference.com/w/cpp/utility/intcmp.

Арифметика з насиченням – C++26

- ```
  template<class T>
    constexpr T add_sat(T x, T y) noexcept;
  template<class T>
    constexpr T sub_sat(T x, T y) noexcept;
  template<class T>
    constexpr T mul_sat(T x, T y) noexcept;
  template<class T>
    constexpr T div_sat(T x, T y) noexcept;
  template<class T, class U>
    constexpr T saturate_cast(U x) noexcept;
  ```
- Детальніше див. "P0543R3: Saturation arithmetic".
  ```

Отримання еквівалентних знакових чи беззнакових типів – C++11

- ```
  template< class T >
  struct make_unsigned;
  template< class T >
  using make_unsigned_t = typename make_unsigned<T>::type;
                                  // C++14

  template< class T >
  struct make_signed;
  template< class T >
  using make_signed_t = typename make_signed<T>::type;
                                  // C++14
  ```

- Використання:

  ```
  using uint_type  = std::make_unsigned_t<int>;
  ```
  ```

Підтримка бітових операцій, <bit> – C++20

- std::bit_width(x) – кількість бітів, потрібна для представлення x, $1 + [\log_2 x]$, або 0 для x == 0.
- Інші: has_single_bit(), bit_ceil(), bit_floor(), bit_width().
- std::rotl() та rotr() – циклічні зсуви ліворуч та праворуч.
- countl_zero(), countl_one(), countr_zero(), countr_one() – порахувати, відповідно, кількість старших/молодших нулів чи одиничок.
- popcount() – порахувати кількість одиничних бітів.
- C++23 додав std::byteswap() – змінює порядок байт на протилежний, наприклад, для зміни endianness. Лише для цілих типів.
- Детальніше див. `https://en.cppreference.com/w/cpp/header/bit` та "<bit> in Visual Studio 2019 version 16.8 Preview 2".
- std::endian дозволяє визначити endianness системи: `https://en.cppreference.com/w/cpp/types/endian`.

std::unreachable() – C++23

- Функція std::unreachable() з <utility> дозволяє анотувати певний код, як недоступний.
  - Наприклад, default в switch, який ніколи не може бути досягнуто.
- Використовується як підказка компілятору.
- Її виклик – невизначена поведінка.

Змінна _ – C++26

- В C++26 заплановано спеціальну змінну _ – для значень, які не планується використовувати.
  - ◇ Позбавляє потреби в [[maybe_unused]] у деяких місцях.
- Різні використання в одній області видимості не конфліктують:

```
std::unique_lock _{list_lock};

auto _ = list.insert(1);
auto _ = list.insert(2);
auto _ = list.insert(3);

auto [_, _, value, _] = list.do_something();

enum MmapBits {
  Shared,
  Private,
  _,
  _,    // OK: пропустити два
  Fixed,
  Rename,
};
```

  - ◇ Але не працює з глобальними змінними.
- Якщо одна в поточній області видимості – поводить себе як звичайна змінна, щоб не ламати зворотну сумісність.
- Оригінальний документ: "A nice placeholder with no name".

Лінійна алгебра: BLAS та LAPACK! – C++26

- Приклад:

```
linalg::matrix_vector_product(A, x, y); // y = A * x

.....
auto A_m = submdspan(A, m, full_extent, full_extent);
auto x_m = submdspan(x, full_extent, m);
auto y_m = submdspan(y, m, full_extent);
// y_m = A * x_m
linalg::matrix_vector_product(A_m, x_m, y_m);
```

- Підтримується, в тому числі, ExecutionPolicy політика, тому можлива паралелізація, включаючи багатопоточну чи SIMD.
- Детальніше див.: "A free function linear algebra interface based on the BLAS" та "Padded mdspan layouts" – про підтримку взаємодії std::submdspan і BLAS.

Перетворення в стрічки

- "C++ String Conversion: Exploring std::from_chars in C++17 to C++26"

std::simd – C++26

- Оригінальний документ: std::simd, але до нього є ще купа доповнень.
- Див. також "SIMD library", приклад звідти:

```
#include <experimental/simd>
namespace stdx = std::experimental;

template<class A>
stdx::simd<int, A> my_abs(stdx::simd<int, A> x) {
    where(x < 0, x) = -x;
    return x;
}

    const stdx::native_simd<int> a = 1;
    const stdx::native_simd<int> b([](int i) { return i - 2; });
    const auto c = a + b;
    const auto d = my_abs(c);
    const auto e = d * d;
    const auto inner_product = stdx::reduce(e);
    const stdx::fixed_size_simd<long double, 16> x([](int i) { return i
        const auto r = stdx::pow(stdx::cos(x), 2) + stdx::pow(stdx::sin
}
```

